

棋局特征提取与解析层深研报告：面向你“象棋AI混合代理系统”的可实现方案

执行摘要

你提供的两份文档把“棋局特征提取解析层”的工程方向定得很清楚：在《详细产品与工程落地文档》中，你计划实现 **FEN→90×14 one-hot→位置编码→4-6层PyTorch TransformerEncoder→输出128-512维特征向量**，并将向量写入Neo4j用于余弦相似检索（Sprint 4“Transformer关键特征提取器/相似局面检索”段落）；同时提出交互体验“响应延迟目标<3秒”。在《推理与中高层解释增强版 v2》中，你进一步把“特征提取器”职责升级为“**语义特征抽取器**”：不仅产出向量，还必须产出可进入 **semantic_state** 的中间变量（阶段、核心矛盾、王安全、子力协调、结构弱点等），并通过 **evidence_map** 实现Claim-证据绑定，新增 **semantic_extractor** 与 **evidence_verifier** 节点，避免LLM凭空创造术语。

本报告给出一个可直接落地的解析层总设计（输入模态、状态Schema、不变量/增强、数据管线、缓存/延迟策略），并提供至少5套可选架构（经典特征工程/ResNet-CNN/Transformer/GNN/NNUE/混合多流），每套都包含：mermaid结构图、张量/图结构与形状、动作空间编码、损失函数与训练策略（监督/自对弈/自监督/弱监督）、算量与延迟/内存取舍、可解释性方案、以及与下游模块（policy/value、搜索、检索、解释器、引擎）集成点。方法依据主线工作：AlphaZero/AlphaGo Zero、KataGo、MuZero、NNUE、GNN与Transformer及自监督学习等。¹

从你的文档提炼的解析层目标与接口边界

你的系统属于“引擎（Pikafish）+知识库/棋书+LLM+视觉+Agent编排”的混合代理。解析层应当位于**输入侧（FEN/棋谱/图片识别）与下游决策/检索/解释之间**，并输出三类产物：

- **Facts（可验证事实特征）**：规则层（合法着法mask、将军/被将、子力清单、重复/步数等）+引擎层（bestmove、score、PV、depth、nodes等）。AlphaZero明确依赖规则生成合法着并对非法着概率mask为0再归一化，且把部分规则信息（如重复计数等）编码进输入特征。²
- **Learned Embedding（可学习表示向量）**：用于相似局面检索（你Sprint 4目标）、也可作为下游policy/value网络或检索条件。
- **Semantic（语义中间变量）**：与你v2一致：把facts再“提炼”为可解释语义标签（多标签）、核心矛盾与候选假设，并绑定证据来源（ENGINE/BOOK/KG/VISION/INFERRED）。

建议的解析层对外接口

- 1) `parse_input(raw) -> Position`：支持FEN、PGN/自定义棋谱、以及视觉识别结果。
- 2) `extract_facts(position) -> facts`：规则引擎+UCI引擎统一产出。
- 3) `encode(position, facts, arch=...) -> {embedding, policy_logits, value, semantic_logits, aux}`
- 4) `augment(position, labels) -> (position', labels')`：对称/扰动与动作同步变换。

5) `pack_for_storage(embedding, semantic, facts_digest)`：写入Neo4j向量属性、并写入LangGraph `semantic_state/evidence_map`。

与引擎集成时，Pikafish明确为“UCI xiangqi engine”，并包含 `.nnue` 网络文件，这意味着解析层必须能输出/消费UCI格式（bestmove、pv等）并可选择复用NNUE评估。³

解析层总体设计：输入模态、状态Schema、不变量与延迟策略

输入模态与统一数据结构

解析层建议把输入拆成三段：

- **Board state**：棋盘格点+棋子类型/颜色；side-to-move；（可选）回合数、重复计数、无进展计数等。AlphaZero把这类规则信息纳入输入特征并在附录中总结。²
- **Move history**：推荐默认走“历史堆叠为通道”（CNN风格），而不是把“历史×格点”展平成Transformer token；因为self-attention复杂度随 N^2 增长，历史token化在大棋盘上会带来明显内存/延迟压力（本对话中已给出 N^2 随K增长的量化图）。
- **Metadata**：引擎配置、时间控制、用户等级、检索到的棋书/图谱摘要等。

状态Schema（与v2语义层对齐）

建议在LangGraph的State中固定以下字段（字段名可按你v2直接落地）：

- `facts.rules`: {legal_moves, legal_mask, check_flags, repetition_count, move_number, material}
- `facts.engine`: {bestmove, score_cp(or wdl), depth, pv, nodes, time_ms}
- `facts.vision`: {fen, pieces, confidence}（如来自图片）
- `semantic_state`: {phase, initiative, king_safety, coordination, weaknesses, move_type, ...}（多标签）
- `reasoning_state`: {core_contradiction, candidate_hypotheses, uncertainties}
- `evidence_map`: {claim_id: [ENGINE|BOOK|KG|VISION|INFERRED]}（由verifier维护）

不变量与增强（rotation/reflection/视角归一）

AlphaZero指出：Go具有旋转/镜像对称，并用于（1）生成8种对称做数据增强，（2）MCTS评估时随机变换再送入网络做“平均化”；而Chess/Shogi规则不对称，一般不做此类增强。⁴

对Xiangqi建议采用“可控增强”：- **side-to-move归一**：把“轮到黑走”的局面做180°旋转+交换红黑，使网络总在“我方=当前行棋方”的坐标系学习（动作与policy也同步变换）。

- **左右镜像**：作为数据增强（相当于交换左/右翼），但解释层术语映射需一致（如“左路/右路攻防”）。

延迟与缓存（满足<3秒交互目标）

- **缓存层级**：`FEN->tensor` 缓存、`FEN->legal_mask` 缓存、`(FEN,engine_depth)->engine_result` 缓存。
- **两级解析**：在线默认走“轻量解析”（规则+浅模型/embedding+必要引擎深度）；用户要求深算或教学解释时再触发“重解析”（更深引擎、更大模型、更长PV、多证据检索）。

- **增量更新**：若采用NNUE路线（见后文D），可做到“走一步只更新少量输入特征”，极适合低延迟CPU推理。

5

可选的特征提取架构方案

下表先对比取舍，随后给出每个架构的可实现细节（≥4套、均含mermaid图）。

架构	表示	主要用途	典型优势	典型风险
A ResNet-CNN (AlphaZero风格)	$B \times C \times H \times W$ 平面堆叠	policy/value + MCTS、通用强基线	成熟、吞吐高、易mask与对称增强	全局关系依赖堆层数/模块
B Transformer (棋盘token)	$B \times N \times d$	生成embedding/语义标签、融合meta	全局交互直接、易接LLM语义头	历史token化会涨 N^2 成本
C GNN (节点/边+edge-policy)	Graph(V,E)	关系推理、跨棋盘泛化	边注意力=可解释“为何此着”	图构建与动作映射复杂
D NNUE (增量浅网)	稀疏二值特征+浅MLP	极低延迟评估/与引擎融合	CPU百万级eval/s目标、易量化	表达力受限、偏评估不偏策略
E 混合多流+语义头	learned+handcrafted+enginePV	你v2“语义层+证据绑定”终态	同时服务检索/解释/决策	多任务权重与工程复杂

A：AlphaZero式ResNet-CNN解析器（最稳的“可训可上搜索”基线）

```

flowchart LR
    IN[Position+History] --> PL[Plane Encoder: BxCxHxW]
    PL --> TR[ResNet trunk]
    TR --> PH[Policy head -> logits HxWxM]
    TR --> VH[Value head -> scalar]
    TR --> SH[Semantic multi-heads]
    PH --> MASK[Mask illegal moves + renorm]
    MASK --> OUTPI["π(a|s)"]
    VH --> OUTV["v(s)"]
    SH --> OUTS[semantic_state candidates]
  
```

动作空间编码（Xiangqi可借鉴AlphaZero的“起点格×动作平面”）

AlphaZero在Chess用 $8 \times 8 \times 73$ 动作平面，在Shogi用 $9 \times 9 \times 139$ ，并mask非法着。

对Xiangqi可定义 $9 \times 10 \times M$ （推荐 $M \approx 59$ ，含直线步数/马/象/士/将/兵类别），总动作约5310，工程上同量级易实现。

输入平面 (Xiangqi示例)

- `piece_planes`: $14 \times K$ (7子 $\times$ 2色, K 为历史步数; 你Sprint 4方案已以“ 90×14 one-hot”作为基础)

- `extras`: side-to-move (若不做视角归一)、重复/回合数等 (可先最小化)

得到: $x \in B \times (14K + \text{extras}) \times 10 \times 9$

损失与优化

KataGo明确给出基础loss包含policy头与value头, 并可叠加更多辅助头, 且训练使用SGD momentum 0.9、batch size 256 (单GPU可容纳上限)。⁷

-
$$L = \text{CE}(\pi_{\text{target}}, \pi_{\text{pred}}) + \lambda v \cdot \text{MSE}(z, v) + \lambda \text{sem} \cdot \text{BCE}(y_{\text{sem}}, p_{\text{sem}}) + \lambda \text{reg} \cdot ||\theta||^2$$

工程落地点

- 训练: PyTorch; 分布式可参照OpenSpiel的AlphaZero实现与接口 (便于快速起跑与自对弈管线)。⁸

- 推理: 在线轻模型 (如 $C=64$, $_{128}$, $\text{blocks}=6$, $_{10}$), 并缓存FEN->tensor、FEN->legal_mask。

B: 棋盘Token Transformer解析器 (与你文档v1的TransformerEncoder最贴近)

```
flowchart LR
    IN[Position/Facts/Meta] --> TOK[Tokenize: squares (+ optional move/meta tokens)]
    TOK --> EMB[Embed + 2D PosEnc/RelPos]
    EMB --> TX[TransformerEncoder L]
    TX --> POOL[CLS/Attn Pool]
    POOL --> VEC[Embedding 128-512]
    TX --> POL[Policy head]
    TX --> SEM[Semantic heads]
```

实现要点

Transformer来自“self-attention”架构。⁹

你v1方案用 `nn.TransformerEncoder` 是可行起步, 但建议两点改造以更贴近棋类:

1) **embedding方式**: 与其对14维one-hot做 `Linear(14->d_model)`, 更常见做法是 `piece_id Embedding` (离散表) + 坐标位置编码 (2D pos或相对位置bias)。

2) **汇聚方式**: 避免简单 `flatten(d_model*90)` 再FC; 更稳的是使用 `[CLS]` token或attention pooling输出全局embedding, 方便迁移到不同棋盘大小 (与你v2强调的“语义层/检索复用”更一致)。

形状 (Xiangqi示例)

- token数: $N=90$ (仅当前局面); $x_{\text{tok}} \in B \times N \times d_{\text{model}}$

- 输出embedding: $B \times 256$ (对应你文档计划“输出128-512维向量”)

- policy输出: 可直接预测 $H \times W \times M$ 动作平面, 或预测“from-square logits + move-type logits”再组合 (更省参数)。

训练目标与集成点

- 检索: 把embedding写入Neo4j向量属性实现“相似局面检索” (你Sprint 4)。

- 解释: semantic head输出多标签语义, 让 `semantic_extractor` 节点更多是“结构填充与证据绑定”, 而不是

从零生成。

- 低延迟：避免把K步历史当token；历史建议仍堆叠在通道或用少量“最近着法token”。（上文图已说明 N^2 随K增长的压力。）

C：GNN解析器（节点/边特征 + “按边输出policy”，更强的关系表达与可解释性）

```
flowchart LR
    IN[Position] --> G[Build Graph: nodes=squares or pieces, edges=moves/attacks]
    G --> GN[GNN layers: GAT/MPNN + edge features]
    GN --> POL[Edge/Move logits]
    GN --> VAL[Global readout -> value]
    GN --> SEM[Semantic heads]
    POL --> MASK[Mask illegal edges]
    MASK --> OUTPI[" $\pi(a|s)$ "]
```

主参考：AlphaGateau（2024）给出了“用图表示Chess并扩展GAT以携带边特征、自然输出policy”的完整范式：

- 它把“棋盘格为节点、着法为有向边”，并明确列出节点特征与边特征设计；节点特征包含棋子one-hot、历史与局面状态信息，边特征包含“该边是否为合法着、位移信息、升变/走子能力”等。¹⁰

- 它基于GAT（Graph Attention Networks）并提出带边特征更新的扩展（GATEAU），用于生成按边的move logit。¹¹

为什么它对你很关键

Xiangqi、Chess、Shogi都不是Go那种纯平移不变格点动作：走子会形成“棋子-格点-可达性”的图关系。GNN的“消息传递”框架能更自然表达这些关系（MPNN是此类模型的统一抽象之一）。¹²

并且，边注意力权重天然可用于解释：“这步棋为何优先探索/为何抑制某些边”，与v2“中高层解释增强”方向高度契合。

Xiangqi的两种图构建落地方案

- **Square-graph（推荐起步）**：90个格点节点；边为“所有可能的相对动作类别”落在棋盘内的有效边（类似AlphaGateau对Chess只保留有效边）。policy直接是边logits，mask非法边。

- **Piece-interaction graph（增强版）**：节点为棋子，边为攻击/防守/牵制/同线关联，边特征含“距离/是否隔子/是否将军线”；更贴近战术解释，但需要更复杂规则抽取。

实现建议

- 框架：PyTorch Geometric 或 DGL（工程上都成熟）。

- 模型：GAT/GINE/MPNN变种；若要边特征可按AlphaGateau的思路扩展attention。¹³

D：NNUE式增量特征网络（为<3秒乃至“毫秒级”评估服务，适合与Pikafish深度融合）

```
flowchart LR
    IN[Position] --> FEAT[Sparse features (e.g., HalfKP-like)]
    FEAT --> ACC[Incremental accumulator update]
    ACC --> MLP[Shallow quantized MLP]
```

```
MLP --> V[value/centipawn or WDL]
V --> OUT[Fast evaluation for search & explanation]
```

关键事实：NNUE的设计目标就是“输入每步只小幅变化→可增量更新→低精度整数域推理→极低延迟CPU评估”。 Stockfish官方文档明确说明NNUE的三原则：输入稀疏、变化小、网络足够简单以便整数域低精度推理；并指出其目标是每线程百万级eval/s且强依赖量化。⁵

为何你应把它纳入解析层选项

- 你系统已经依赖Pikafish，而Pikafish分发包含 `.nnue` 网络文件并强调其为强力评估。³
- 如果你的在线“复盘解释/对弈”需要严格延迟预算，NNUE可作为“事实层快速评估/候选着筛选”的第一层，再把少量关键候选交给更重的模型与LLM解释。

Xiangqi特征集如何仿照HalfKP落地

Stockfish NNUE最常用HalfKP特征：`(king_square, piece_square, piece_type, piece_color)`并基于此实现增量更新。⁵

Xiangqi可定义“HalfKG”（以将/帅位置为条件）或“HalfKP-90”（把64格扩展到90格），训练目标可用：

- 引擎评估分数（cp）回归；或
- WDL分类（win/draw/loss），与解释器“胜率”更一致。

工程实现路线

- 训练：PyTorch（稀疏输入/自定义batch loader）；
- 推理：C++/SIMD + int8/int16量化（可直接参考Stockfish nnue-pytorch文档中对量化与层类型的讨论）。¹⁴

E：混合多流 + 语义头（你v2“语义层/证据绑定/解释闭环”的终态架构）

```
flowchart LR
    POS[Position] --> RULES[Rule facts: mask/attacks/material]
    POS --> NET[Backbone: CNN or Transformer or GNN]
    RULES --> FUSED[Fusion MLP]
    NET --> FUSED
    ENGINE[Engine PV/score] --> FUSED
    FUSED --> EMB[Embedding for retrieval]
    FUSED --> POL[Policy head]
    FUSED --> VAL[Value/WDL head]
    FUSED --> SEM[Ontology multi-label heads]
    SEM --> SEMSTATE[semantic_state + core_contradiction candidates]
    SEMSTATE --> VERIF[evidence_verifier]
```

研究依据：KataGo证明“在AlphaZero式框架上增加辅助头与全局汇聚能显著提升效率与强度”：其论文总结了包括全局池化（global pooling）与多个辅助目标（ownership/score等）的改进，并给出了训练细节（SGD momentum 0.9、batch 256、policy/value基础loss可叠加更多项）。⁷

对你而言，这是把“语义层”做成**可训练的中间监督**最直接的证据：semantic head不必100%依赖人工标注，可以大量使用弱监督/伪标签（见下一节）。

语义监督的三种强落地方式（从易到难）

- 1) **规则弱监督**：如“是否将军/是否牵制/是否兑子/是否弃子”等可规则判定标签；先覆盖教材中最常见概念。
- 2) **引擎诱导伪标签**：用PV差异与评估变化生成“forcing_move / improving_move / blunder”等标签（你本体里有多标签要求）。KataGo也使用“辅助目标”思想来提升训练效率。⁷
- 3) **LLM辅助标注 + verifier清洗**：LLM产生候选语义标签与解释，但必须由 `evidence_verifier` 按“ENGINE/BOOK/KG/INFERRED”来源打标并过滤（这是你v2的核心约束）。

训练与监督策略：标签类型、增强、损失、数据需求与库选型

标签类型（建议你按“可获得性”分层）

- **policy监督**：
 - 自对弈MCTS根访问分布 π （AlphaZero标准做法），或引擎soft target（top-k + 温度）。AlphaZero用MCTS产生 π 并用其训练策略网络。¹⁵
 - **value监督**：终局胜负 z （或WDL），与policy一起训练（AlphaZero输出 (p, v) 并用其引导搜索）。¹⁶
- **aux/semantic监督**：规则可判定 + 引擎伪标签 + 少量人工金标（用于评测与校准）。

自监督/半监督（降低语义标注成本）

- **Masked modeling（棋盘版BERT）**：随机mask棋子/格点，让模型预测被mask内容（BERT即以“masked预训练”著称）。¹⁷
- **对称一致性对比学习**：把同一局面的对称变换视作“正样本对”，用contrastive loss拉近embedding、拉远不同局面；SimCLR总结了对比学习框架与“增强组合很关键”。¹⁸
- **BYOL式无负样本一致性**（可选）：用于稳定学习embedding再微调到检索/语义分类。¹⁹

数据来源与工程管线

- **自对弈数据**：PXZero/PikaXiangqiZero生态提供象棋神经网络引擎与训练相关基础设施与参数公开页面，可作为你搭建自对弈数据管线的参考。²⁰
- **棋谱/库**：Wukong Xiangqi项目包含PGN解析与棋局数据库工具链，可用作“棋谱解析/开局书/题库生成”的工程参考。²¹
- **引擎真值**：Pikafish为UCI引擎，可批处理生成 `(fen, bestmove, score, pv)` 作为监督或评测集事实层。³

推荐库/框架（按任务）

- 规则与合法着：自研象棋规则引擎（保证mask一致）+UCI通信模块。
- 深度学习：PyTorch（你文档选型一致）；CNN/Transformer直接用 `torch.nn`。
- GNN：PyTorch Geometric/DGL（若走C）。
- 推理加速：ONNX Runtime/TensorRT（CNN/Transformer）；NNUE走C+++量化与SIMD（按Stockfish文档）。⁵

评估与实验计划：数据集、指标、消融与基线

你的v2强调“没有评测就不知道系统更会分析还是更会说”，因此解析层必须配套最小可行benchmark（MVP）并能持续迭代。

数据集划分（建议）

- **Train**：大规模棋谱/自对弈 positions（去重、按开局/中局/残局分布采样）。
- **Dev**：固定局面集（覆盖常见战术主题与语义概念）。
- **Test**：三类：
 - 1) **Move prediction test**：给定局面预测top-1/top-k是否命中引擎bestmove或人类着法；
 - 2) **Value calibration test**：预测胜率/WDL与引擎/终局结果的一致性；
 - 3) **Semantic解释评测集**：按你v2“概念本体”标注规范构建，评估多标签F1与证据绑定正确率（ENGINE/BOOK/KG/INFERRED不混淆）。

指标（务必同时覆盖“强度、稳定、可解释”三轴）

- **policy**：top-1 / top-3 / top-5命中率；policy交叉熵；在MCTS中作为先验时的nps-固定下胜率提升。
- **value**：MSE/Huber；WDL accuracy；校准误差（ECE，分桶）。
- **强化对弈强度**：Elo（自对弈或与固定引擎/旧版本对弈）。AlphaZero以Elo尺度分析搜索与思考时间可扩展性，并对比MCTS与alpha-beta引擎搜索速度差异。⁴
- **检索**：Recall@K（相似局面检索能否找回同一开局/相近结构的局面）；embedding聚类纯度。
- **语义与证据**：
 - 多标签macro/micro-F1；
 - **evidence_map** 准确率（“ENGINE事实”不得被INFERRED冒充，这是你v2的硬约束）。

消融实验（建议最少做这6个）

- 1) 输入：仅当前局面 vs 加K步历史通道。
- 2) 动作空间：from×plane编码（推荐） vs flat move list（AlphaZero提到对Chess/Shogi flat也可但训练略慢）。⁶
- 3) Backbone：CNN vs Transformer（同参量预算）。
- 4) GNN：无边特征 vs 有边特征（参考AlphaGateau）。¹⁰
- 5) 语义头：无语义头 vs 加语义多任务（看对policy/value与解释指标的提升）。
- 6) 缓存/增量：普通评估 vs NNUE增量评估（在线延迟与吞吐对比）。⁵

关键伪代码：解析与编码的“最小可行实现骨架”

```
# 1) 统一局面对象
class Position:
    board: list[list[int]] # H×W piece_id (0=empty)
    side_to_move: int      # 0/1
    move_number: int
    repetition_count: int
```



```

    history: list["Position"] # last K positions (optional)

# 2) FEN -> Position (Xiangqi: 9×10)
def parse_fen_xiangqi(fen: str) -> Position:
    # TODO: split board / side / counters; decode piece letters -> ids
    # return Position(...)

# 3) 规则生成合法着 mask (关键：与动作编码严格一致)
def legal_mask_xiangqi(pos: Position, action_encoder) -> "np.ndarray[H,W,M]
bool":
    mask = zeros((10, 9, M), dtype=bool)
    for move in generate_legal_moves(pos):
        idx = action_encoder.move_to_index(move)
        mask[idx.h, idx.w, idx.m] = True
    return mask

# 4) 平面编码：B×C×H×W (历史作为通道堆叠)
def encode_planes(pos: Position, K=8) -> "np.ndarray[C,H,W]":
    planes = []
    for t in range(K):
        p = pos.history[t] if t < len(pos.history) else pos
        planes.append(one_hot_piece_planes(p.board, num_piece_planes=14)) #
14×H×W
    extras = extra_rule_planes(pos) # e.g., side_to_move, repetition, etc.
    return concat(planes + extras, axis=0)

# 5) 模型前向：同时产出embedding / policy / value / semantic
def forward(model, pos):
    x = encode_planes(pos) # C×H×W
    mask = legal_mask_xiangqi(pos, action_encoder) # H×W×M
    policy_logits, value, embedding, sem_logits = model(x)
    policy = masked_softmax(policy_logits, mask)
    return policy, value, embedding, sem_logits

```

以上骨架的关键工程点是：**动作编码与legal_mask必须绑定为同一个“ActionEncoder”实现**；否则任何对称增强、policy训练都会错位。

(注：你要求“无单独参考文献列表”，因此本文只做行内引用；所有关键方法依据的主线论文/项目已在段落内给出出处链接引用标记。)

¹ ² ⁴ ⁶ ¹⁵ ¹⁶ <https://arxiv.org/pdf/1712.01815.pdf>
https://arxiv.org/pdf/1712.01815.pdf?utm_source=chatgpt.com

³ [GitHub - official-pikafish/Pikafish: UCI xiangqi engine](https://github.com/official-pikafish/Pikafish)
https://github.com/official-pikafish/Pikafish?utm_source=chatgpt.com

5 14 NNUE | Stockfish Docs

https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html?utm_source=chatgpt.com

7 [1902.10565] Accelerating Self-Play Learning in Go

https://ar5iv.org/pdf/1902.10565?utm_source=chatgpt.com

8 AlphaZero — OpenSpiel documentation

https://openspiel.readthedocs.io/en/latest/alpha_zero.html?utm_source=chatgpt.com

9 [PDF] Attention Is All You Need by Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin · 10.48550/arxiv.1706.03762 · OA.mg

https://oa.mg/work/10.48550/arxiv.1706.03762?utm_source=chatgpt.com

10 11 13 [2410.23753] Enhancing Chess Reinforcement Learning with Graph Representation

https://ar5iv.org/pdf/2410.23753?utm_source=chatgpt.com

12 Neural Message Passing for Quantum Chemistry

https://proceedings.mlr.press/v70/gilmer17a?utm_source=chatgpt.com

17 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding - arXiv.gg

https://arxiv.gg/abs/1810.04805?utm_source=chatgpt.com

18 SimCLR: Contrastive Visual Representations

https://www.emergentmind.com/papers/2002.05709?utm_source=chatgpt.com

19 Bootstrap your own latent: A new approach to self-supervised Learning - Science Explorer Abstract

https://scixplorer.org/abs/2020arXiv200607733G/abstract?utm_source=chatgpt.com

20 PXZero

https://px0.org/training_runs?utm_source=chatgpt.com

21 GitHub - maksimKorzh/wukong-xiangqi: Didactic Chinese chess Xiangqi engine by Code Monkey King

https://github.com/maksimKorzh/wukong-xiangqi?utm_source=chatgpt.com